Objects and Classes

Definitions we will learn!

- What is an object?
- What is a class?
- Abstraction
- Mutators
- Accessors
- Private Helpers
- Initialization
- Constructor
- Constructor overloading

JAVA OOP

- OOP stands for **Object-Oriented Programming**.
- Procedural programming is about writing procedures or methods that perform operations on the data, while object-oriented programming is about creating objects that contain both data and methods.
- Object-oriented programming has several advantages over procedural programming:
  - OOP is faster and easier to execute
  - OOP provides a clear structure for the programs
  - OOP helps to keep the Java code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug
  - OOP makes it possible to create full reusable applications with less code and shorter development time.

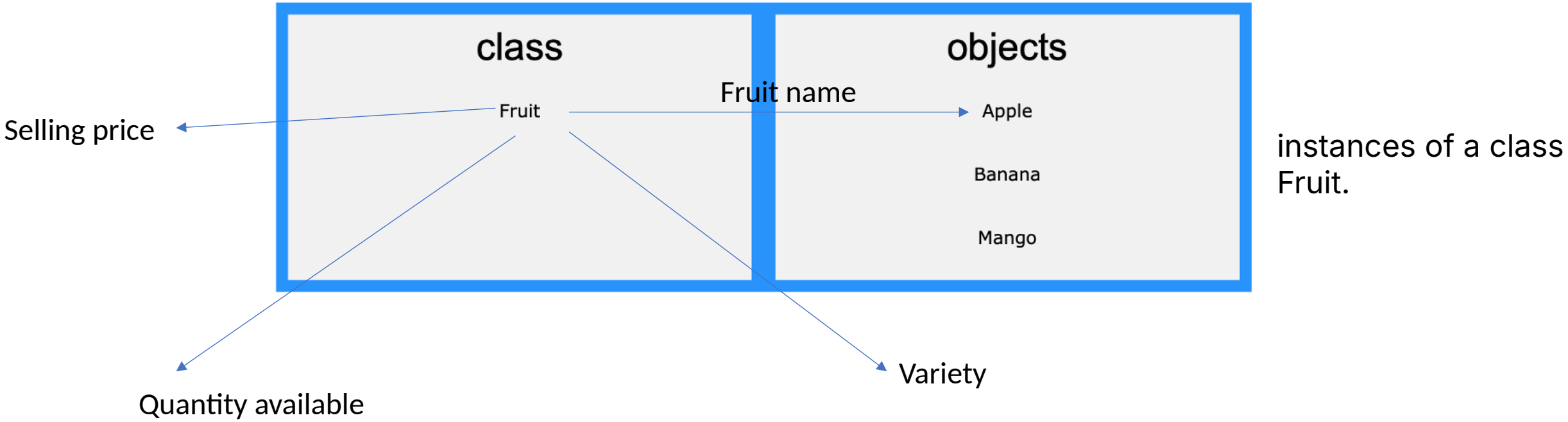
Object and Class

class	objects
Fruit	Apple
	Banana
	Mango

Another example:

class	objects
Car	Volvo
	Audi
	Toyota

Objects



A class is a template for objects, and an object is an instance of a class.
When the individual objects are created, they inherit all the variables and methods from the class.

Abstraction

Abstraction means to have a user interact with an item at a high-level, with lower-level internal details hidden from the user (aka ***information hiding*** or ***encapsulation***).

Real life examples of abstraction:

A car presents an abstraction to a user, including a steering wheel, gas pedal, and brake.

A refrigerator presents an abstraction to a user, including freon gas, a compressor, and a fan.

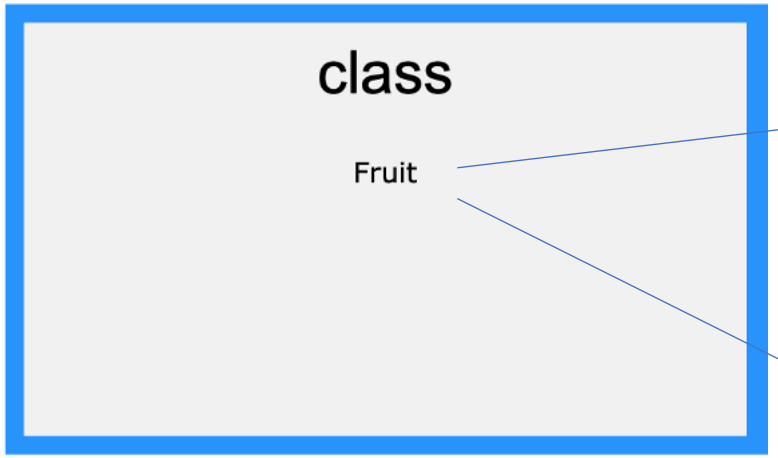
In Java Programming we achieve abstraction using classes.

Objects strongly support abstraction, hiding entire groups of methods and variables, exposing only certain methods to a user.

Classes

Everything in Java is associated with **classes** and objects.

attributes **methods**



1. Fruit name
2. Variety
3. Current selling price
4. Quantity available

Attributes or variables

1. Increase selling price
2. Check if product in stock

methods

Class

class	objects
Fruit	Apple
	Banana
	Mango

Another example:

class	objects
Car	Volvo
	Audi
	Toyota

Create a class

```
public class Main {
int x = 5;
}
```

```
public class Main {
  int x = 5;

  public static void main(String[] args) {
    Main myObj1 = new Main();  // Object 1
    Main myObj2 = new Main();  // Object 2
    System.out.println(myObj1.x);
    System.out.println(myObj2.x);
  }
}
```

Create a class Method

Class

```
static void hello() {
    System.out.println("Hello");            // An object cannot access it
}                                           // Can be accessed directly ClassName.hello()

public void hello2() {
    System.out.println("Hello");            // An object can access this
}
```

Code snippet from main program

```
Fruit1.hello(); //this is error. Cannot access non-static method.
Fruit1.hello2(); // this is valid

Inventory.hello();  // this is valid. This is how you can access static methods.
Inventory.hello2(); //this is error. You need to have an instance of class Inventory
```

Static vs. Non-Static

You will often see Java programs that have either static or public attributes and methods.

Mutators and Accessors

Mutators and accessors

A class' public methods are commonly classified as either mutators or accessors.

- A ***mutator*** method may modify ("mutate") a class' fields.
- An ***accessor*** method accesses fields but may not modify a class' fields.

```
static void hello() {
    System.out.println("Hello");              // Accessor
}

public void setName(String FruitName) {
    FruitType = FruitName;                     //Mutator
}

public void setQuantity(int NewItems) {
    FruitQuantity = NewItems;                  //Mutator
}
```

Private Helpers

A programmer commonly creates private methods, known as ***private helper methods***, to help public methods carry out tasks.

```
    private void increaseQuantity2(){
    FruitQuantity =FruitQuantity +1;
}
    public void increaseQuantity(int number){
        for (int i=0;i<number;i++)
        increaseQuantity2();
    }
```

1.In addition to public member methods, a class may define private member methods.
2.Any member method (public or private) may call a private member method.
3.A user of the class can call public member methods, but a user can not call private member methods (which would yield a compiler error).

Initialization

A good practice is to initialize all variables when declared. This section deals with initializing the fields of a class when a variable of the class type is allocated.

```
public class Inventory {

    private String FruitType = "some Fruit";
    private int FruitQuantity = -1;
    private float FruitPrice = -1;
```

By default, Java would initialize all the values to NULL.

Constructor

Java provides a special class member method, known as a ***constructor***, that is called when an object of that class type is created, and which can be used to initialize all fields. A constructor that can be called without any arguments is called a ***default constructor***

```
public class Inventory {

    private String FruitType;
    private int FruitQuantity;
    private float FruitPrice;

    public Inventory(){
        FruitType = "some fruit";
        FruitQuantity = -1;
        FruitPrice = -1;
    }
```

Not defining any constructor is essentially the same as defining a constructor with no arguments.

Constructor Overloading

A class creator can ***overload*** a constructor by defining multiple constructors differing in parameter types. When an object is created with the new operator, arguments can be passed to the constructor. The constructor with matching parameters will be called.

```
public Inventory(){
    FruitType = "some fruit";
    FruitQuantity = -1;
    FruitPrice = -1;
}

public Inventory(String Fruitname){
    FruitType = Fruitname;
    FruitQuantity = -1;
    FruitPrice = -1;
}

public Inventory(String Fruitname, int NewItems){
    FruitType = Fruitname;
    FruitQuantity = NewItems;
    FruitPrice = -1;
}
```

ALWAYS HAVE A DEFAULT CONSTRUTOR

```java
public class Inventory {

    private String FruitType;
    private int FruitQuantity;
    private float FruitPrice;
```

IS SAME AS

```java
public class Inventory {

    private String FruitType;
    private int FruitQuantity;
    private float FruitPrice;

    public Inventory(){
        FruitType = "NULL";
        FruitQuantity = 0;
        FruitPrice = 0;
    }
```

Not defining any constructor is essentially the same as defining a constructor with no arguments.

```java
public class Inventory {

    private String FruitType;
    private int FruitQuantity;
    private float FruitPrice;
```

IS NOT SAME AS

```java
public Inventory(String Fruitname){
    FruitType = Fruitname;
    FruitQuantity = -1;
    FruitPrice = -1;
}
```

JAVA WILL NOT CREATE DEFAULT CONSTRUCTOR. WILL RESULT IN COMPILER ERROR

In-Class Activity – Bank Account

```
public class Main {
    public static void main(String[] args) {
        Account One = new Account ();
        One.setName("Tom");
        One.setBalance(1000);

        One.printDetails (); // Should print "Hi Tom, You have $1000"

        One.Withdraw(100);

        One.printDetails (); // Should print "Hi Tom, You have $900"

        One.deposit(500);

        One.printDetails (); // Should print "Hi Tom, You have $1400"

        Account Two = new Account ();
        Two.printDetails (); // Should print "Hi NoName, You have $0"

    }
}
```

Write a class Account that will ensure the code snippet to work as desired.

All printDetails() call should work and there should be no compiler error.

You can either use initialization or constructor.

In-Class Activity – Solution

```java
public class Account {
    private String Name ="NoName";
    private double Balance = 0;

    public void setName(String NewName) {
        Name = NewName;
    }

    public void setBalance(double NewBal) {
        Balance = NewBal;
    }

    public void printDetails() {
        System.out.println("Hi "+ Name +", You have $" + Balance);
    }

    public void deposit(double depositCash) {
        Balance = Balance + depositCash;
    }

    public void Withdraw(double WithdrawCash) {
        Balance = Balance - WithdrawCash;
    }
```

You can also have a solution with constructors.